CLOUD-NATIVE LLM ORCHESTRATION PLATFORM

COSMOS

Your depth-first roadmap from senior DevOps engineer to Cloud Solution Architect — 14 stages, built one running increment at a time.

- **56** weeks
- ~ **13** months
- **8–10** hrs / week
- **14** stages
- **30+** ADRs
- **3** ⭐ anchors

PROGRESS **0%** 0 / 56 weeks · Start at Stage 0.5

Collapse all | Reset

Deep-new — your genuine gaps | Extend — adjacent to what you know | Leverage — you already operate this | Mixed | ⭐ Go-to-90% anchor

A

PHASE A

Wks 1–7

Language & app discipline

0/2

Stage 0.5 Wks 1–4 DEEP

Python + testing foundations

The #1 gap — application Python, not scripting Python. Everything stands on this.

WHAT YOU BUILD

A typed, tested async service module with a real DB migration — no distributed systems yet.

CONCEPTS TO OWN

- typing + Pydantic v2
- generators/coroutines vs async/await
- context managers & decorators
- pytest: fixtures, parametrize, mocking
- hypothesis property tests
- async SQLAlchemy + Alembic
- uv / packaging & venv hygiene

READ FIRST

- Fluent Python — data-model + async chapters
- FastAPI + SQLAlchemy async tutorial
- pytest docs

Done when · You can write a typed async module with a migration and a meaningful test suite — and explain the event loop aloud.

Stage 0 Wks 5–7 DEEP

FastAPI rewrite + clean architecture

Rewrite the CRUD to FastAPI with layered, testable structure.

WHAT YOU BUILD

FastAPI rewrite; hexagonal (ports/adapters); validation; consistent errors; correlation IDs; idempotency keys; optimistic concurrency; 12-factor config.

CONCEPTS TO OWN

- hexagonal / ports & adapters
- idempotency vs at-least-once
- optimistic vs pessimistic concurrency
- 12-factor config
- correlation IDs / request context

READ FIRST

- Fundamentals of Software Architecture
- 12factor.net
- Cloud Design Patterns: Idempotency, Health Endpoint Monitoring

B

PHASE B

Wks 8–14

The distributed core

0/2

Stage 1 Wks 8–11 DEEP

Async core: queue, state machine, event sourcing

Accept jobs instantly; process them durably.

WHAT YOU BUILD

202 + id on submit; durable queue (Redis Streams); worker; explicit job state machine; append-only event log; at-least-once + idempotent consumer; cancellation.

CONCEPTS TO OWN

- separating accept from process
- state machines in code
- event sourcing → state reconstruction
- idempotent consumers
- the exactly-once myth

READ FIRST

- DDIA Ch.1, 5, 9
- Cloud Design Patterns: Queue-Based Load Leveling, Competing Consumers, Event Sourcing, CQRS

🎓 CKA — start background prep here (nearly free for you)

Stage 2 Wks 12–14 EXTEND

Polyglot persistence: MongoDB + Redis

The right datastore for each shape of data.

WHAT YOU BUILD

Relational core stays in Postgres; trajectories/payloads → MongoDB; Redis hot state. Write the “why polyglot” ADR.

CONCEPTS TO OWN

- embed vs reference modeling
- aggregation pipeline
- change streams
- TTL indexes

READ FIRST

- DDIA Ch.2
- MongoDB Data Modeling + 6 Rules of Thumb
- MongoDB University M001 / M320

🎓 CKA — start background prep here (nearly free for you)

C

The agent layer ⭐

PHASE C
Wks 15–19

0/1

Stage 3 Wks 15–19 DEEP ☆ Emergent core

Agent loop + RAG

The Emergent core — a real agent loop with Claude. Fully new; non-negotiable time.

WHAT YOU BUILD

A job type running plan → tool-call → observe → iterate, bounded by a token budget; persist the trajectory; stream via SSE; multi-agent patterns; RAG (embeddings + pgvector/Qdrant); an eval harness.

CONCEPTS TO OWN

- safe agent-loop termination
- tool / function calling
- context-window & token budgeting
- embeddings & semantic retrieval
- when RAG helps vs hurts
- multi-agent patterns

READ FIRST

- Anthropic API docs — Messages, tool use
- Anthropic — Building Effective Agents

D

PHASE D
Wks 20–24

Resilience ☆

0/1

Stage 4 Wks 20–24 DEEP ★ Anchor #1

Rate-limit / breaker / retry / fallback

Turn SRE intuition into resilience you built. Highest interview value.

WHAT YOU BUILD

Provider registry; selector (round-robin/weighted/least-loaded); rate limiter (token bucket + sliding window via Redis/Lua); circuit breaker (closed/open/half-open); retry + backoff + jitter; fallback chain; load shedding.

CONCEPTS TO OWN

- why jitter matters
- half-open probing
- when fallback is an anti-pattern
- load-shedding vs backpressure
- distributed rate limiting

READ FIRST

- Release It! — stability patterns
- Amazon Builders' Library — retries/backoff/jitter, load shedding, avoiding fallback

Done when · Your story: "I've operated these in Envoy — here's how I built them."

E

PHASE E

Wks 25–29

Event-driven backbone

0/1

🏁 **End of the spine.** Weeks 1–29 carry ~60% of your hours — this is where the real transformation happens.

Stage 5 Wks 25–29 DEEP

Kafka + CQRS + Saga + CDC

An event-streaming backbone with real guarantees. End of the spine.

WHAT YOU BUILD

Kafka (Redpanda locally); partitions/consumer-groups/replay/exactly-once; DLQ; schema registry + evolution; outbox pattern; CQRS read models; Saga for multi-step jobs; CDC (Debezium) Postgres→warehouse.

CONCEPTS TO OWN

- partitions & consumer groups
- exactly-once semantics
- outbox pattern
- CQRS
- Saga: orchestration vs choreography
- change data capture

READ FIRST

- DDIA Ch.11
- Confluent Kafka 101
- microservices.io: Saga, Transactional Outbox
- Debezium docs

Done when · Own everything to here (Week 29) and you're already a different candidate.

F

PHASE F

Wks 30–32

Multi-tenancy & identity

0/1

Stage 6 Wks 30–32 EXTEND

Tenants, identity, Vault

Many tenants, real identity, real secrets. New = app-side auth logic.

WHAT YOU BUILD

Tenants, API keys, quotas, usage ledger; OAuth2/OIDC/JWT (Keycloak); Vault dynamic creds + PKI; RBAC; prompt-injection defense.

CONCEPTS TO OWN

- OAuth2 / OIDC / JWT flows
- tenant isolation & fair scheduling
- dynamic secrets & PKI
- quota / usage ledgers
- prompt-injection defense

READ FIRST

- Well-Architected Security pillar
- Keycloak docs
- Vault docs — dynamic secrets, PKI
- OWASP API Security Top 10

G

PHASE G
Wks 33–36

Internal comms & Go

0/1

Stage 7 Wks 33–36 MIXED

gRPC, service mesh, Go

Polyglot microservices over gRPC + mesh. Go + gRPC are the real learning; the mesh is yours.

WHAT YOU BUILD

gRPC + Protobuf between services; rewrite the worker in Go (goroutines/channels/context); Istio for mTLS + traffic-shaping.

CONCEPTS TO OWN

- gRPC + Protobuf
- Go concurrency: goroutines, channels, context
- service-mesh mTLS
- deliberate traffic shaping

READ FIRST

- A Tour of Go + Effective Go
- gRPC docs + Protobuf guide
- Istio / Linkerd docs

H

PHASE H

Wks 37–40

K8s deep + custom Operator ⭐

0/1

Stage 8 Wks 37–40 LEVERAGE ⭐ Anchor #2

K8s deep + custom Operator + KEDA

Your K8s mastery, elevated to authoring an Operator. Spend the weeks almost entirely here.

WHAT YOU BUILD

Production K8s (RBAC/NetPol/PSS/PDB you already own) + a custom Operator + CRD (Kubebuilder) + KEDA autoscaling on Kafka lag + Kyverno/OPA policy.

CONCEPTS TO OWN

- controllers & the reconcile loop
- CRD design
- KEDA event-driven autoscaling
- admission control / policy-as-code

READ FIRST

- Kubernetes docs — CRDs, controllers
- Kubebuilder / Operator SDK book
- KEDA docs
- Kyverno / OPA Gatekeeper docs

Done when · “I wrote a Kubernetes operator in Go” is a top-tier differentiator you reach fast.

I

PHASE I

Wks 41–44

Frontend + A/B framework

0/1

Stage 9 Wks 41–44 MIXED

Dashboard + experimentation framework

A dashboard + a real experimentation framework.

WHAT YOU BUILD

Next.js/React/TS dashboard; live trajectory (SSE); experimentation framework: feature flags, traffic splitting, statistical significance, guardrails/auto-rollback.

CONCEPTS TO OWN

- React / Next / TypeScript basics
- SSE / streaming UI
- feature flags & traffic splitting
- statistical significance & guardrails

READ FIRST

- Next.js docs
- react.dev Learn
- TypeScript handbook
- Trustworthy Online Controlled Experiments (Kohavi)

J

PHASE J

Wks 45–47

Preview environments

0/1

Stage 10 Wks 45–47 EXTEND

Per-tenant preview environments

Ephemeral per-tenant stacks — powered by your Stage-8 operator.

WHAT YOU BUILD

On-demand isolated stacks (vcluster); external-dns + cert-manager for domains + TLS; lifecycle/teardown; ArgoCD ApplicationSets.

CONCEPTS TO OWN

- vcluster / namespace-per-tenant isolation
- external-dns + cert-manager
- GitOps ApplicationSets
- environment lifecycle & teardown

READ FIRST

- vcluster docs
- external-dns + cert-manager docs
- ArgoCD ApplicationSets docs

K

PHASE K

Wks 48–52

Cloud, IaC, multi-region

0/1

Stage 11 Wks 48–52 MIXED

Terraform AWS + multi-region

Real cloud, real networking, multi-region. Terraform mechanics are yours; multi-region reasoning is new.

WHAT YOU BUILD

Terraform a real AWS env (VPC/EKS+IRSA/RDS/ElastiCache/MSK/S3/ALB/Route53/CloudFront/ACM/W multi-region (active-passive → active-active); FinOps (Infracost); one real deploy.

CONCEPTS TO OWN

- VPC / subnets / SG vs NACL
- IRSA & IAM least-privilege
- active-passive vs active-active
- FinOps: budgets, tagging, cost allocation

READ FIRST

- AWS VPC + IAM best-practices
- AWS EKS Workshop (IRSA/VPC)
- Terraform AWS modules + Terratest
- Well-Architected Reliability + Cost

🎓 AWS SAP-C02 — aim for the Professional here

L

PHASE L

Wks 53–56

Observability, SRE, chaos & capstone ⭐

0/1

Stage 12 Wks 53–56 LEVERAGE ☆ Anchor #3

Instrument, SLOs, chaos, supply-chain, capstone

Instrument what you built; run it like an SRE. “I run Grafana” → “I set this system's SLOs.”

WHAT YOU BUILD

OTel across async/Kafka/gRPC; Prometheus + recording rules/Alertmanager; Loki; Grafana; Pyroscope; SLOs + burn-rate alerts; chaos (Chaos Mesh); supply-chain (SBOM/cosign/SLSA); k6 load test; capstone artifacts.

CONCEPTS TO OWN

- instrumenting your own code (OTel)
- SLO / SLI / error budgets
- burn-rate alerting
- chaos engineering
- supply-chain security (SBOM/cosign/SLSA)

READ FIRST

- Google SRE Book + Workbook
- OpenTelemetry docs
- Chaos Mesh docs
- sigstore/cosign + SLSA docs
- k6 docs

Done when · Capstone: Well-Architected review, cost model, C4 + Exceptional-4 diagrams, failure playbook, scale-at-100x.

Spine first · **own Stages 0.5→5** and the back half rides your existing infra muscle. Read before you build · design in prose · you write 100% of the code · review + ADR. · **Cosmos**